FETCH

The analytics: explanation and computation

How the forecast is worked out, step by step, with every calculation shown

---

This document explains what the predictive part of the system does and then **works each calculation through with numbers**, so that any figure on the Stock planning screen can be traced by hand.

Two kinds of numbers appear, and each is labelled:

- **Worked examples** use small made-up sales histories, so the arithmetic fits on a page. Each was run through the system's own code, and the steps shown here were checked to give exactly the code's answer.
- **Real figures** come from running the system's own forecast on the shop's sales (28 February to 10 October 2026), reading only.

11 October 2026. This matches the code on the work branch after the QA fixes. The settings and results are those in QA/FORECAST-METHOD.md.

1. The whole computation at a glance

The question: how many of each product will sell on each of the next 30 days?

For every product, in this order:

1. **Build its daily history**: items sold on each day, a zero where nothing sold.
2. **Leave out days without data**: days the shop has no usable record for.
3. **Cap freak days**: one extreme day counts for no more than a cap.
4. **Choose the method** by how many days the product has on record: a cautious estimate (under 14 days), a simple trend (14 to 41), or the two machine-learning models (42 or more).
5. **Correct for the last miss**: multiply by a factor worked out from how far off the forecast was in the most recent test.
6. **Turn the forecast into decisions**: how long the stock lasts, when to order, how much.

The numbers the computation uses

Every setting, in one place. Each is explained in the section named.

Setting	Value	Section
Days forecast ahead	30	1
History read	up to 365 days	2
Empty days in a row that count as "closed"	3 or more	3
Cap on one day	the larger of 5, and 4 × the typical selling day (75th percentile)	4
Selling days a product needs to set its own cap	5	4
New product	under 14 days on record	6
Simple trend reads	the last 56 days	7
Simple trend mix	Holt 50%, regression 30%, weighted average 20%	7
Holt smoothing	$\alpha = 0.35$, $\beta = 0.15$, damping $\phi = 0.88$	7
Slow product (under 1 a day)	85% its 28-day average, 15% the mix	7
Floor	55% of the last 14 days' average	7
Model needs	42 days on record (28 + 14)	8
Model reads	the last 120 days; a day's weight halves every 60 days	8
Random Forest	60 trees, depth up to 8, at least 20 rows per leaf	8

Setting	Value	Section
Gradient Boosting	80 trees, depth 2, learning rate 0.05, at least 60 rows per leaf	8
Model's share of the final figure	1 (the simple trend has none, for a product with 42 days)	8
Correction	half of the last miss, kept between 0.6 and 1.4, by category (20 items minimum)	9
Delivery time	7 days	11
"Reorder soon"	stock lasts 21 days or less	11
Safety stock	1.65 × spread × √7	11
Too much stock	more than 90 days of sales	11

2. Step 1: the daily history

The system reads every **completed** sale of the last 365 days (a voided sale is not counted) and adds up, for each product, how many items sold on each day.

- A "day" is the shop's own calendar day (Philippine time). Sales are stored in world time and converted, so a sale at 7 in the morning is not filed under the day before.
- The history starts on the product's first sale and runs to **yesterday**. Today is left out: the day is not over.
- A day with no sale of that product is written as **0**.

Worked example

A product was first sold on 5 October. Its sale lines:

Date	Sale lines	Daily history
5 October	2 items, then 1 item	3
6 October	none	0
7 October	1 item	1
8 October	none	0
9 October	none	0
10 October	4 items (one sale later voided: not counted)	0
11 October (today)	2 items	left out

The history is [3, 0, 1, 0, 0, 0]: 6 days on record.

Real figures. The shop has 2,338 completed sales with 4,473 lines, 10,370 items in all, from 28 February to 10 October 2026.

3. Step 2: days without data

A zero normally means "nothing sold", and the forecast should learn from it. Two kinds of empty day mean something else:

- sales were recorded that day, but none of them has its items
- three or more days in a row with no sale at all: the shop was closed, or nothing was being recorded

The rule

1. Find each run of days on which no item was sold in the whole shop.
2. If any day in the run has a completed sale without items, leave the whole run out.
3. Otherwise, if the run is 3 days or longer, leave it out.
4. A run that reaches the last day on record is kept: recent silence may be real.

Real figures

The rule finds one run in the shop's records: **10 to 31 May 2026, 22 days**.

Days	What is on record	Verdict
10 to 27 May	199 sales (#1097 to #1295), ₱1,259,362.00, none with items	sales without items
28 to 31 May	no sales at all	same run of empty days
The whole run	22 days	left out of every product's history

Those 22 days are skipped when the history is read. A product's "last 28 days" then means its last 28 days on record. **Nothing in the database is changed.**

4. Step 3: the cap on freak days

One model learns from all products together. One freak day in one product (a mistyped quantity, a single bulk purchase) used to bend every product's forecast. So before anything is learned, each day is held to a cap.

```
cap = the larger of 5, and 4 × the typical selling day
```

The **typical selling day** is the 75th percentile of the product's selling days: the amount it reaches or stays under on three of every four days it sells. It is found like this:

```
position = whole part of (0.75 × (number of selling days − 1)), counting from 0
```

Worked example 1

A product's last 30 days: 0, 1, 0, 0, 2, 0, 1, 0, 0, 0, 1, 3, 0, 0, 2, 0, 0, 1, 0, 40, 0, 2, 0, 0, 1, 0, 1, 0, 0, 2.

```
Selling days only, in order:  1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3, 40      (12 selling days)
Position:  whole part of 0.75 × (12 − 1) = whole part of 8.25 = 8
           counting from 0, that is the 9th value:  2
Typical selling day:  2
Cap:  the larger of 5 and 4 × 2 = 8   →   8
The day of 40 is counted as 8. Total goes from 57 to 25.
```

Why the 75th percentile and not the average: the average of those selling days is 4.75, dragged up by the 40 itself. The percentile is not moved by one extreme day.

Worked example 2: too few selling days

A product with fewer than 5 selling days cannot say what is typical for it, so the shop's figure is used. History: 0, 0, 3, 0, 0, 0, 500, 0, 0, 1 (3 selling days). If the shop's typical selling day is 2:

```
Cap:  the larger of 5 and 4 × 2 = 8
History after the cap:  0, 0, 3, 0, 0, 0, 8, 0, 0, 1
```

Real figures

On the shop's data the cap touches 6 of 3,971 selling days. Adding a made-up product with one day of 10,000 used to raise the other 120 products' forecast by 130%. Now it moves them by 4.1%, against 3.8% for the same product without the freak day.

5. Step 4: which method forecasts the product

Days on record, from its first sale	Method	Marked on the page as
Under 14	Cautious first estimate (section 6)	new
14 to 41	Simple trend (section 7)	trend
42 or more	The two machine-learning models (section 8)	model

Why 42: the model needs 28 days to build its inputs (it looks back as far as 28 days) and at least 14 more to learn from.

If the machine-learning program cannot run, every product is forecast by the simple trend and the page says so.

6. The computation for a new product

A daily rate cannot be read from a few days. So what the product has sold so far is spread over 14 days, however few days it has existed.

```
daily rate = items sold so far ÷ 14

forecast for 30 days = daily rate × 30
```

Worked example 1

A product sold 5 on its first day, yesterday.

```
Daily rate:  5 ÷ 14 = 0.3571   (shown as 0.36 a day)
30 days:     0.36 × 30 = 10.8
Range:       from 0 up to twice the rate, 0.71 a day
The old way: 5 a day × 30 = 150, with an order suggested for 165
```

Worked example 2

A product with 6 days on record: 2, 0, 1, 0, 0, 3.

```
Sold so far:  6
Daily rate:   6 ÷ 14 = 0.4286   (shown as 0.43)
30 days:      0.43 × 30 = 12.9
```

The estimate grows as more is sold. On day 14 it equals the plain average, and the simple trend takes over. No correction factor is applied to a new product.

7. The computation of the simple trend

Used for a product with 14 to 41 days on record. It reads the product's last 56 days at most and mixes three ordinary methods.

The three parts

Part A: Holt's smoothing (50%)

It keeps two running figures: a **level** (where sales are now) and a **trend** (which way they are moving). Each new day nudges both.

```
level = 0.35 × today's sales + 0.65 × (yesterday's level + yesterday's trend)
trend = 0.15 × (level − yesterday's level) + 0.85 × yesterday's trend
```

To look i days ahead, the trend is carried forward but **damped**, so a decline flattens out instead of running to zero:

```
Holt value = level + trend × (0.88 + 0.88² + … + 0.88^i)
```

Part B: linear regression (30%)

A straight line is fitted through the days by least squares, with the first day numbered 0. The line is carried forward at most 14 days.

```
slope = (n × Σ(day × sales) − Σday × Σsales) ÷ (n × Σday² − (Σday)²)
intercept = (Σsales − slope × Σday) ÷ n
regression value = slope × (n − 1 + the smaller of i and 14) + intercept
```

Part C: weighted moving average (20%)

The last 7 days, with weights 1 to 7, so the most recent day counts 7 times as much as the day a week ago.

```
weighted average = (1×d₁ + 2×d₂ + … + 7×d₇) ÷ 28
```

Putting them together

1. **Mix:** 0.5 × Holt + 0.3 × regression + 0.2 × weighted average.
2. **Slow product** (average under 1 a day): 0.85 × its average over the last 28 days + 0.15 × the mix. On a product that sells now and then, trend methods mostly follow noise.
3. **Floor:** never below 0.55 × its average over the last 14 days.
4. **Day of the week:** multiply by that weekday's index (below). Used only with 14 days or more on record.

```
weekday index = 1 + 0.5 × (that weekday's average ÷ overall average − 1), kept between 0.3 and 2
```

A weekday seen fewer than twice has an index of 1.

Worked example: a steady seller

28 days on record, ending Saturday 10 October: 3, 1, 2, 2, 1, 3, 4, 3, 2, 1, 2, 2, 3, 5, 2, 2, 1, 3, 2, 3, 4, 3, 1, 2, 2, 2, 4, 5. Total 70, average 2.50 a day.

Part A

Holt starts with level = the first day (3) and trend = second day − first day (1 − 3 = −2), then steps through the days:

Day	Sold	Level	Trend
2	1	1.0000	−2.0000
3	2	0.0500	−1.8425
4	2	−0.4651	−1.6434
28	5	3.5452	0.1730

```
Day 2:  level = 0.35 × 1 + 0.65 × (3 + (−2)) = 0.35 + 0.65 = 1.0000
        trend = 0.15 × (1.0000 − 3) + 0.85 × (−2) = −0.30 − 1.70 = −2.0000
After day 28:  level = 3.5452,  trend = 0.1730
One day ahead:  3.5452 + 0.1730 × 0.88 = 3.6974
```

Part B

```
Slope = 0.03503 a day,  intercept = 2.0271
One day ahead:  0.03503 × (27 + 1) + 2.0271 = 3.0079
```

Part C

```
Last 7 days:  3, 1, 2, 2, 2, 4, 5
(1×3 + 2×1 + 3×2 + 4×2 + 5×2 + 6×4 + 7×5) ÷ 28 = 88 ÷ 28 = 3.1429
```

Together, for Sunday 11 October

```
Mix:    0.5 × 3.6974 + 0.3 × 3.0079 + 0.2 × 3.1429 = 3.3797
Slow?   average 2.50 is not under 1, so the mix stands
Floor:  0.55 × 2.5714 (last 14 days' average) = 1.4143; the mix is above it
Sunday: 4 Sundays sold 11, average 2.75; index = 1 + 0.5 × (2.75 ÷ 2.50 − 1) =
1.05
Forecast:  3.3797 × 1.05 = 3.5487 items
```

The same is done for each of the 30 days; they add up to 116.3. That is above the recent 2.5 a day because this product's last days were its best (4 and 5), and the trend carries that upward. It shows how quickly the simple trend reacts.

Weekday	Days seen	Sold	Index
Sun	4	11	1.05
Mon	4	6	0.80
Tue	4	6	0.80
Wed	4	9	0.95
Thu	4	7	0.85

Weekday	Days seen	Sold	Index
Fri	4	13	1.15
Sat	4	18	1.40

Worked example: a slow seller

28 days: 0, 0, 1, 0, 0, 0, 2, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 2, 0, 0, 0, 1, 0, 0, 1. Total 10, average 0.3571 a day, which is under 1.

```
Mix (as above):  0.5 × 0.4970 + 0.3 × 0.4603 + 0.2 × 0.3929 = 0.4652
Slow product:    0.85 × 0.3571 + 0.15 × 0.4652 = 0.3734
Floor:           0.55 × 0.3571 = 0.1964; not needed
Sunday index:    it never sold on a Sunday; 1 + 0.5 × (0 − 1) = 0.50
Forecast for Sunday 11 October:  0.3734 × 0.50 = 0.1867 items
```

Its 30 days add up to 10.2. A fraction such as 0.19 does not mean a fifth of an item will sell: it means about one sale every five such days.

8. The machine-learning models

Used for a product with 42 days or more on record. Two models from scikit-learn (a standard library for Python) each give a figure, and the two are averaged.

What a model learns from: rows

The models learn from **rows**. Each row pairs what was known on one day with what the product sold some days later.

	What it holds
12 inputs	What kind of day is being forecast, the product's recent sales, its recent level, and how far ahead the day is
1 answer	What the product really sold on that day

For each product, every day after its first 28 becomes one row. The gap between "what was known" and "the day sold" runs through 1, 2, 3 … up to 30 days in turn, so the model learns to forecast any distance ahead. All products' rows go into one pile: **one model for the whole shop.**

Recent rows count for more

```
weight of a row = 0.5 ^ (how many days ago ÷ 60)
```

How long ago	Weight
Yesterday	1.00
30 days	0.71
60 days	0.50
90 days	0.35
120 days	0.25

The model reads only the last 120 days on record of each product.

The 12 inputs, worked for one row

A made-up product, 100 days on record, ending Saturday 10 October. Its last 28 days: 1, 0, 0, 2, 3, 0, 1, 0, 0, 1, 2, 3, 1, 0, 0, 1, 0, 2, 3, 0, 0, 1, 0, 0, 2, 3, 0, 1. The forecast wanted is for **Tuesday 13 October**, 3 days ahead.

Input	Value	How it is found
Day of the week	2	Tuesday (Sunday counts as 0)
Weekend	0	no
Day of the month	13	

Input	Value	How it is found
Month	10	October
Public holiday	0	no
Sales on the last day on record	1	Saturday 10 October
Sales a week before the first day forecast	1	Sunday 4 October, a week before 11 October
Sales two weeks before it	0	Sunday 27 September
Sales four weeks before it	1	Sunday 13 September
Average of the last 7 days	1	7 sold in 7 days
Average of the last 28 days	0.9643	27 sold in 28 days
How far ahead	3	days

Nothing among the inputs says which product it is. What tells the model how much this product sells is its own recent sales and averages.

How each model turns a row into a number

A decision tree

A chain of yes-or-no questions about the inputs ("Is the 28-day average above 0.5?", "Is it a weekend?"). Each path ends in a **leaf**, and the leaf's answer is the weighted average of what was sold in the training rows that reached it. The tree picks, at each step, the question that splits the rows into the two most alike groups.

Random Forest: many trees, averaged

```
Random Forest = (tree 1 + tree 2 + … + tree 60) ÷ 60
```

Each tree is grown on a different random sample of the rows, and at each question may choose among only 3 of the 12 inputs (the square root of 12, rounded down). The trees therefore differ, and one odd day misleads a few of them, not most. A leaf must hold at least 20 rows.

Gradient Boosting: small corrections, one after another

```
start = the average of all answers

after each tree:  figure = figure + 0.05 × (that tree's estimate of what is
still wrong)
```

80 small trees (two questions deep, at least 60 rows per leaf), each grown on a random 80% of the rows. Each looks only at the mistakes the earlier ones left and corrects 5% of them. Small steps keep it from memorising.

The two together

```
forecast for a day = (Random Forest + Gradient Boosting) ÷ 2, never below 0
```

Worked example: a small fit

To show real numbers, the system's own model was fitted on 10 made-up products with 100 days each (720 rows). For the row above (Tuesday 13 October):

```
Random Forest:       0.589
Gradient Boosting:   0.684
Average:             (0.589 + 0.684) ÷ 2 = 0.636 items   (rounded from the
unrounded figures)
```

Each of the 30 days is forecast this way, straight from the last real sales. Nothing the model predicts is fed back in as if it had happened.

	Next 30 days
Random Forest alone	20.35
Gradient Boosting alone	23.60
Both, averaged	**21.98**
For comparison: what this made-up product sold in its last 30 days	27

In this small fit the forest leaned most on: average of the last 28 days (29%), average of the last 7 days (24%), day of week (11%), how far ahead (7%). On the Stock planning page the same list is shown for the real fit.

> **The same sales always give the same forecast.** Both models use a fixed starting number for their random choices, so a re-run reproduces the figures exactly.

The mix with the simple trend

```
final = (1 − share) × simple trend + share × model
```

The share is **1**: for a product with 42 days or more, the model's figure is used alone. It was 0.5 before the re-tune. Section 13 says how 1 was chosen.

9. The last step: the correction

Every method reads a product's level from its recent weeks. When sales are falling the forecast runs high; when they are rising it runs low. So the system measures how far off it was in its most recent test and corrects **half** of that.

```
factor = 1 + 0.5 × (sold ÷ forecast − 1)
```

- The factor is kept between **0.6 and 1.4**.
- It is worked out **for each category**. A category that sold fewer than 20 items in the test fortnight is too thin to judge and takes the whole shop's factor.
- It uses only days already past.
- It is not applied to a new product.

```
final forecast = forecast × factor of its category
```

Real figures: the factors used for 27 September to 10 October

They come from the fortnight before, **13 to 26 September**: what the method forecast for those days (using only sales up to 12 September) against what sold.

Category	Forecast	Sold	Sold ÷ forecast	Factor
Food	84.96	65	0.765	**0.883**
Health & Wellness	61.96	74	1.194	**1.097**
Accessories	31.96	32	1.001	**1.001**
Litter & Waste	7.20	4	0.556	**0.970** (shop's)
Whole shop	186.08	175	0.940	**0.970**

```
Food:             1 + 0.5 × (65 ÷ 84.96 − 1) = 1 + 0.5 × (0.765 − 1) = 0.883
Health & Wellness:  1 + 0.5 × (74 ÷ 61.96 − 1) = 1 + 0.5 × (1.194 − 1) = 1.097
Accessories:        1 + 0.5 × (32 ÷ 31.96 − 1) = 1 + 0.5 × (1.0013 − 1) = 1.0006,
shown as 1.001
Litter & Waste:     sold 4, fewer than 20, so it takes the shop's factor
Whole shop:         1 + 0.5 × (175 ÷ 186.08 − 1) = 0.970
```

These are exactly the factors the system applied. Their effect on the next fortnight:

Category	Forecast before the correction	After	Sold
Food	86.4	76.2	61
Health & Wellness	64.2	70.5	64
Accessories	29.2	29.2	32
Litter & Waste	6.5	6.3	3
Whole shop	**186.3**	**182.2**	**160**

The correction brought Food closer (86.4 to 76.2, with 61 sold) and moved Health & Wellness away (64.2 to 70.5, with 64 sold). It helps on balance, not every time: over the four test fortnights it took the forecast from 27.3% high to 15.6% high.

10. The range, the trend arrow and confidence

These describe the product. They are worked out from its last 56 days and are not part of the forecast figure.

The range on the chart

```
low = forecast − 1.28 × spread        high = forecast + 1.28 × spread
(low is never below 0)
```

"Spread" is the standard deviation of the product's daily sales: the usual distance of a day from the average. The band is a rough guide. It has not been tested as an 80% interval and should not be called one.

The trend arrow

"Up" when the fitted line rises by more than 2% of the average per day, "down" when it falls by more than that, otherwise "stable".

```
Steady seller above:  slope 0.03503, 2% of average = 0.050   →   stable
```

Confidence

```
score = volume + fit + steadiness, kept between 5 and 95
```

Part	Marks	Formula
Volume	up to 35	35 × (days looked at ÷ 30), at most 35
Fit	up to 30	30 × R^2 of the straight line (0 = no fit, 1 = perfect)
Steadiness	up to 35	35 × (1 − (spread ÷ average) ÷ 2), not below 0. **Zero when nothing sold**

70 or more reads **high**, 45 or more **medium**, under 45 **low**.

Worked examples

Product	Volume	Fit	Steadiness	Score	Reads
the slow seller above	32.67	0.27	4.54	37	low
the steady seller above	32.67	1.92	27.03	62	medium
a product with no sale in 8 weeks	35.00	0.00	0.00	35	low

```
Slow seller:  28 days → 35 × 28 ÷ 30 = 32.67;   R² 0.0089 → 0.27;   spread ÷
average = 1.74 → 35 × (1 − 1.74 ÷ 2) = 4.54
              32.67 + 0.27 + 4.54 = 37.48 → 37
No sale in 8 weeks:  35 + 0 + 0 = 35.  Before the fix it scored 35 + 0 + 35 = 70:
"high".
```

Real figures. All 119 products that have sold now read "low" (scores 35 to 39): no product sold on more than 13 of the last 56 days.

11. From the forecast to a stock decision

```
daily = 30-day forecast ÷ 30
```

Figure on the page	Computation
Stock lasts	stock ÷ daily, rounded down
Runs out on	today + that many days
Out	stock is 0
Reorder now	stock ≤ reorder level, or stock lasts 7 days or less
Reorder soon	stock lasts 21 days or less
Order by	today, if stock ≤ reorder level; otherwise run-out date − 7 days
Suggested order	the larger of (forecast + 7 × daily − stock) and (2 × reorder level − stock), rounded up, less what is already on order
Safety stock	1.65 × spread × √7, rounded up
Reorder point	7 × daily, rounded up, + safety stock
Too much stock	stock − 90 × daily, when stock lasts more than 90 days
Likely to expire	stock − daily × days until expiry, rounded up

The 7 is the delivery time in days, taken to be the same for every supplier. The 1.65 gives about a 95% chance of not running out during the wait, if sales vary as they have.

Worked example 1: a product to reorder

Forecast 30 for 30 days. 12 in stock. Reorder level 10. Spread of daily sales 1. Today is 11 October.

```
Daily:            30 ÷ 30 = 1.00
Stock lasts:      12 ÷ 1.00 = 12 days   →   runs out on 2026-10-23
Status:           12 is above the reorder level (10); 12 days is more than 7 and
no more than 21   →   Reorder soon
Order by:         2026-10-23 − 7 days = 2026-10-16
Suggested order:  larger of (30 + 7 × 1.00 − 12 = 25) and (2 × 10 − 12 = 8)   →
25
Safety stock:     1.65 × 1 × √7 = 1.65 × 2.646 = 4.37   →   5
Reorder point:    7 × 1.00 + 5 = 12
```

The page would read: recommended reorder point 12, which includes a safety stock of 5 (it is currently set to 10).

Worked example 2: too much stock

Forecast 6 for 30 days. 60 in stock. Cost ₱50 each.

```
Daily:         6 ÷ 30 = 0.20
Stock lasts:   60 ÷ 0.20 = 300 days, more than 90
Excess:        60 − 90 × 0.20 = 60 − 18 = 42 items
```

```
Tied up:       42 × ₱50 = ₱2,100
```

Worked example 3: likely to expire

30 in stock, forecast 15 for 30 days, expiry in 20 days.

```
Daily:             15 ÷ 30 = 0.50
Sells by expiry:   0.50 × 20 = 10
Left on that day:  30 − 10 = 20 items likely to expire unsold
```

Fast, steady, slow

Among products expected to sell at least 0.5 in 30 days: the top quarter read "fast", the bottom quarter "slow", the rest "steady". A product expected to sell under 0.5 with nothing sold in the last 30 days reads "not selling".

12. Measuring how good the forecast is

The test

1. Hide the last 14 days on record.
2. Forecast them from the days before, with the correction the forecast would have had at the time.
3. Compare with what sold.

The system does this by itself each time the forecast is worked out. Nothing from the hidden days is used to make the forecast.

The measures

Measure	Formula	Says
MAE	average of |forecast − sold|	The usual size of a miss, in items
RMSE	√(average of (forecast − sold)²)	The same, with large misses counting for more
MAPE	average of |forecast − sold| ÷ sold, where sold > 0	The miss as a percentage of what sold
WAPE	Σ|forecast − sold| ÷ Σsold	The total miss as a percentage of the total sold
Bias	(Σforecast − Σsold) ÷ Σsold	Whether it leans high (+) or low (−)
Page accuracy	100 − |Σforecast − Σsold| ÷ Σsold × 100	How close the shop total was

Worked example: five products, one fortnight

Product	Forecast	Sold	Miss	Size of miss	Squared	% of sold
A	3.2	2	+1.2	1.2	1.44	60.0%
B	0.8	0	+0.8	0.8	0.64	cannot be worked out
C	1.5	4	-2.5	2.5	6.25	62.5%
D	0.4	1	-0.6	0.6	0.36	60.0%
E	2.1	2	+0.1	0.1	0.01	5.0%
Total	**8.0**	**9**		**5.2**	**8.70**	

```
MAE:            5.2 ÷ 5 = 1.04 items
RMSE:           √(8.70 ÷ 5) = √1.74 = 1.32 items
MAPE:           (60.0 + 62.5 + 60.0 + 5.0) ÷ 4 = 46.9%   (on the 4 products that sold)
WAPE:           5.2 ÷ 9 = 57.8%
```

```
Bias:           (8.0 − 9) ÷ 9 = -11.1%   (leans low)
Page accuracy:  100 − |8.0 − 9| ÷ 9 × 100 = 88.9%
```

Read the last two lines together. The shop total was 88.9% accurate, while the usual miss on one product was 1.04 items, more than half of what a product sold. Misses in opposite directions cancel in a total. Accuracy and error are both true, and they are different things: always say which one a figure is.

Real figures

Four fortnights the forecast was not shown, 16 August to 10 October 2026. 120 products in each: 480 cases, 780 items sold.

	MAE	RMSE	MAPE	WAPE	Bias	Forecast against sold
Before the fixes	1.549	1.917	73.8%	95.3%	+36.4%	1,064.3 against 780
Now	**1.402**	**1.741**	**59.9%**	**86.3%**	**+15.6%**	**901.4 against 780**
Now, before the correction step	1.475	1.811	62.7%	90.8%	+27.3%	992.9 against 780

Fortnight	Forecast	Sold	Bias	Page accuracy
16 to 29 August	305.5	235	+30.0%	70.0%
30 August to 12 September	254.7	210	+21.3%	78.7%
13 to 26 September	159.1	175	−9.1%	90.9%
27 September to 10 October	182.2	160	+13.9%	86.1%

```
Page accuracy, 27 September to 10 October:  100 − |182.2 − 160| ÷ 160 × 100 = 100 − 13.9 = 86.1%
```

Way of forecasting	MAE	RMSE	Bias
The system now	**1.402**	**1.741**	**+15.6%**
The two models alone	1.472	1.810	+26.8%
Simple trend only	1.582	2.188	+24.9%
Rule of thumb: the average of the last 28 days	1.577	2.137	+28.1%
Rule of thumb: the same as the last 14 days	1.648	2.422	+17.8%
Forecast nothing at all	1.625	2.422	−100%

The system is 11% better than the best rule of thumb on MAE ((1.577 − 1.402) ÷ 1.577) and 19% better on RMSE.

What single products look like

The six best sellers of 27 September to 10 October, forecast against sold:

Product	Category	Before correction	Factor	Forecast	Sold
GOATS MILK 100G	Health & Wellness	1.95	× 1.097	2.14	8
WHISKAS KITTEN	Food	2.05	× 0.883	1.81	6
BEARING SHAMPOO 300ML	Accessories	2.17	× 1.001	2.17	6
ST. ROCHE SHAMPOO	Accessories	2.00	× 1.001	2.00	6
PETYUM	Food	0.93	× 0.883	0.82	5
1 BOX SMP	Health & Wellness	2.02	× 1.097	2.22	5

Each was forecast at about 1 to 2 and sold 5 to 8. That fortnight they sold more than their recent weeks suggested, while 51 other products were forecast a little and sold nothing. This is what "an average miss of 1.4 items" looks like, and why the forecast is of most use for a category or the shop, not for one product.

13. How the settings were chosen

The settings in section 1 were not picked by eye. 1,620 combinations were tried: how fast older days fade (never, 30, 60 days) × how far back the model reads (365, 120 days) × the simple trend's floor and slow window (6 choices) × the model's share (0, 0.25, 0.5, 0.75, 1) × the kind of last step (9 choices).

```
3 × 2 × 6 × 5 × 9 = 1,620 combinations
```

The rule, written down before any result was seen:

1. Keep the combinations whose forecast total is within 10% of what sold.
2. Among those, take the one with the smallest RMSE.

Walk-forward: each test fortnight was scored with the combination the rule picked from earlier fortnights only, so no fortnight was scored with settings fitted to it.

Test fortnight	Earlier windows used	Old settings: MAE / bias	Chosen: MAE / bias
16 to 29 August	5	1.788 / +41.4%	1.763 / +42.9%
30 August to 12 September	7	1.601 / +33.4%	1.561 / +15.8%
13 to 26 September	9	1.417 / +30.2%	1.214 / −5.2%
27 September to 10 October	11	1.325 / +26.5%	1.262 / +13.9%
All four		**1.533 / +33.7%**	**1.450 / +18.9%**

> **The figure to quote is +18.9% bias and MAE 1.450**: the result for settings chosen without sight of the fortnight they were scored on. The +15.6% and 1.402 in section 12 are for the system as it now stands, on fortnights that also helped choose its final settings.

14. What the computation cannot do

Figures for made-up products in this section are the forecast before the correction step (section 9), which would then multiply each by its category's factor.

Limit	Why
The error per product is as large as the sales	A product sells one or two items a fortnight. Even "forecast nothing" misses by 1.63
It still runs high, by about 19%	Sales have fallen every fortnight since July, and any method that reads recent weeks lags a fall
A lasting rise is recognised slowly	The model has learned from this shop that a burst usually fades. A made-up product that went from 1 sale every 5 days to 1 a day was forecast 6.2 after 20 days, 14.5 after 42 and 25.1 after 84, against a true 30
A product in sharp decline can be kept too high	With under 42 days on record the simple trend's floor holds it up: 20 a day falling to 0 over 20 days is still forecast 113
An empty shelf looks like no demand	The computation reads sales. It is not told when a product was out of stock
Seasons and moving holidays	225 days cannot show a year. Holy Week, promotions and price changes are not inputs
The settings come from a falling period	All 13 tuning windows are from July to October 2026. They have not been tested while sales rise
Fixed assumptions	7 days' delivery for every supplier, and a 95% service level, are set, not learned

15. Where each computation is in the code

Computation	File	Function
Daily history	backend/routes/predictions.js	buildDailySeries
Days without data	backend/utils/forecastGuards.js	noDataDays
The cap	backend/utils/forecastGuards.js	dailyCap, capSeries
New product	backend/utils/forecastGuards.js	cautiousDailyRate
Simple trend	backend/routes/predictions.js	simpleTrend, holtSmoothing, linearRegression, weightedMovingAverage, dayOfWeekIndices
The 12 inputs and the two models	ml/forecast.py	_features, _rows, fit_group, forecast_group
Days read and their weights	backend/utils/mlForecast.js	FIT_WINDOW, HALF_LIFE
The order of the steps and the mix	backend/routes/predictions.js	forecastSeries, TUNING
The correction	backend/utils/forecastAdjust.js	errorFactors, factorFor
Confidence	backend/routes/predictions.js	computeConfidence
Stock decisions	backend/routes/predictions.js	planStock
The accuracy test	backend/routes/predictions.js	holdoutTest, backtestAccuracy
The error measures	scripts/forecast-backtest.js	metrics
Choosing the settings	scripts/forecast-tune.js	

To repeat the real figures (both read the database and write nothing):

```
node scripts/forecast-backtest.js 2026-10-10 2026-09-26 2026-09-12 2026-08-29
node scripts/forecast-tune.js
```